ShuffleSnap: strided windowed exact reassignment for point-to-grid layout

Kyle McDonald

October 6, 2026

Abstract

Giving each point of a 2D embedding its own grid cell with the least total squared displacement is a linear assignment problem too large for dense solvers at 10^5 points. ShuffleSnap, inspired by Shellsort, solves small exact assignment problems in windows whose cells lie a stride apart, and shrinks the stride to one. Against certified optima it comes within hundredths of a percent on clustered layouts, including real UMAP embeddings, in seconds, closer than BSP-OT and RasterFairy on most inputs. Its errors are loops of one-cell shifts that no window can see, which make uniform points, concave masks and surplus cells hard.

1 Introduction

An image collection is often shown by embedding the items in the plane, for example with t-SNE or UMAP [6, 11], and placing each thumbnail in its own grid cell. Rounding every point to its nearest cell causes collisions and holes, so the cells must be assigned jointly: we seek the assignment σ of points p_i to distinct cells $c_{\sigma(i)}$ that minimizes $C(\sigma) = \sum_i \|p_i - c_{\sigma(i)}\|^2$. This is a linear assignment problem, and a discrete optimal-transport problem [8]. Dense exact solvers [2, 4] need the full cost matrix, 80 GB at 10^5 points. Practical tools instead split the cloud recursively, like RasterFairy [3], or merge randomized partitions, like BSP-OT [1].

ShuffleSnap borrows from Shellsort [10], which sorts interleaved subsequences at a shrinking gap. It solves only small assignment problems, but exactly, with their cells spread across the grid first and brought together later (Figure 1). We measure its gaps against a solver that proves optimality on the same instances, and report where it fails as well as where it works.

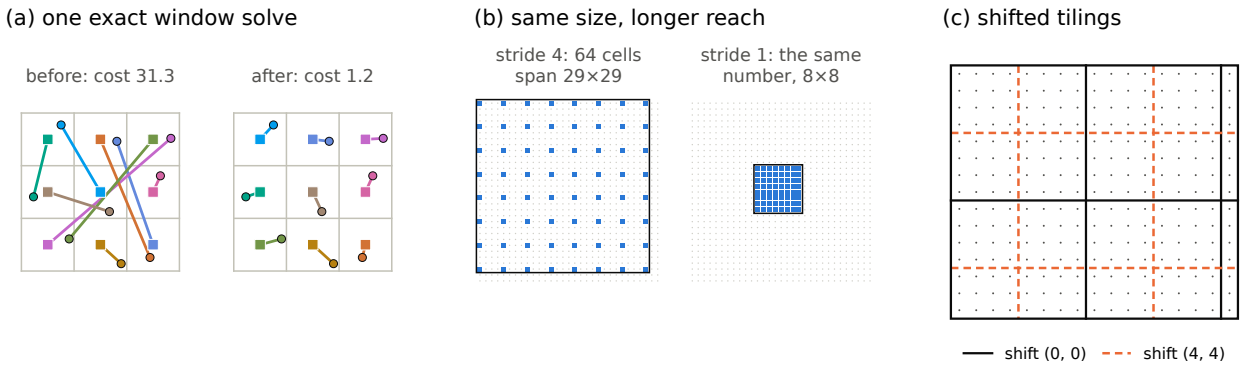


Figure 1: (a) A window collects the points that occupy its cells and reassigns them among the same cells by an exact linear assignment. (b) At stride s a window takes every s-th row and column: it keeps 64 cells but spans $7s + 1$ cells per axis, so points can travel far. (c) Each subgrid is tiled four times, shifted by half a window, so that points can cross every window border.

2 The method

ShuffleSnap improves a valid assignment in place:

1. **Start** from a seeded random permutation of the points over the cells.
2. **Stride.** At stride s, cells whose row and column have the same residues modulo s form one of s^2 interleaved subgrids. Each subgrid is cut into windows of 8×8 of its cells, which span $7s + 1$ cells of the grid per axis.
3. **Solve.** In every window, the current occupants are reassigned among the window's cells, empty ones included, by an exact linear assignment on squared distances to the original coordinates. The result replaces the old assignment only if it is cheaper by more than a relative 10^{-12}.
4. **Shift.** A round tiles every subgrid four times, shifted by $(0, 0)$, $(0, 4)$, $(4, 0)$ and $(4, 4)$. Windows that cross an edge are clipped, never dropped.
5. **Schedule.** On an axis of length L the first stride is the smallest power of two with $8s \geq L$, so that the first windows span the grid. After each round the stride is divided by $\sqrt{2}$ and rounded; at stride 1, rounds repeat until one moves no point.

Because the old assignment is feasible for every window, the total cost never increases, the layout is valid after every step, and ties cannot cause cycling. The windows of one tiling are disjoint, so they run in parallel with a result that does not depend on the number of threads. Each level costs about the same, linear in the number of cells. The final no-move round is a fixed point of the windows, not a proof of optimality.

Rectangular grids use one stride per axis, masked cells never enter a window, and empty cells do, so with more cells than points the solver also chooses which cells stay empty. The kernel is C++ with OpenMP and a Python interface, needs about 55 bytes per point, and skips windows whose occupants have not changed. It solves each window exactly from starting duals derived in closed form from per-axis optimal transport, so that the points of a distant cluster do not all compete for one corner cell. A 70-line serial version in NumPy and SciPy reproduces its assignments exactly. We chose the window side and the stride ratio on development instances. Larger windows ended closer to the optimum in every test, but took longer to reach any cost that a smaller window also reached. Each step from 8×8 to 12×12 to 16×16 cost 2–2.6 times the time and removed one half to two thirds of the remaining excess. We offer *fast* (5×5), *balanced* (8×8, default) and *quality* (12×12); fast and balanced take about as long as BSP-OT with 16 and 64 plans, so the two compare at equal time. Dividing the stride by $\sqrt{2}$ was the best or close to the best ratio at every window size.

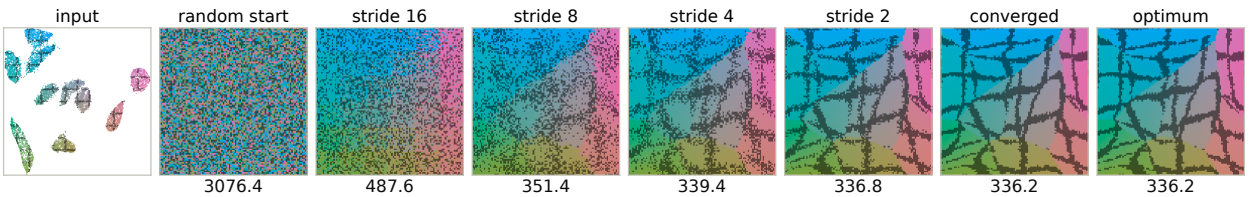


Figure 2: Progress on 10,000 points of a UMAP embedding of MNIST digits [5]. Every point keeps the color of its input position, and the dark bands are graph-paper lines drawn on the input: both travel with the points (every second stride shown). The coarse strides sort the layout globally, stride 1 polishes it. Numbers are the mean squared displacement in cells^2, next to the certified optimum.

3 Certified optima

Gaps need a true optimum, but at 10^5 points no dense solver fits in memory. For any cell potentials v, however,

$$\mathrm{LB}(v) = \sum_{i=1}^{N} \min_j \big(\|p_i - c_j\|^2 - v_j\big) + \sum_{j=1}^{M} v_j - (M - N) \max_j v_j$$

is a lower bound on the optimal cost, by weak duality of the assignment linear program, where $M \geq N$ is the number of cells; it is tight for optimal potentials. Our solver therefore works on a sparse problem and checks it against the dense one. It solves a min-cost flow on candidate edges (each point's nearest cells and the neighbourhood of its cell in a starting layout) with OR-Tools [7], computes potentials from that solution by shortest paths, evaluates LB over *all* cells, adds every edge that the bound shows to be missing, and repeats until the bound meets the cost. The result is then the exact optimum of the dense problem; the start affects only the time. This is the core-and-price idea of sparse assignment and transport solvers [9, 12]. Wherever SciPy's dense solver fits, both found the same optimum: on 600 instances of up to 2,500 points and on all ten distributions at 10^4 points. Started from ShuffleSnap's layout, it certified all 202 benchmark instances up to 2×10^5 points, and uniform instances up to 10^6.

4 Setup

All runs used one workstation (Intel i9-12900KS, 24 hardware threads). Single-thread runs were pinned to one performance core; every run was a fresh process, and times include each method's own preparation but not generating the instance. Design decisions used one set of instances, and all results below use three held-out ones. The inputs are ten synthetic distributions with 10^5 points on near-square grids (Table 1), the complete UMAP embeddings of MNIST and Fashion-MNIST [13] with 70,000 points each, and masks. Every method received identical coordinates and cells. We compare BSP-OT through its official Python binding with 16 and 64 merged plans, and RasterFairy in its current upstream version, which solves leaf blocks exactly and is far better than its 2019 PyPI release. Table 1 lists the results for all ten distributions.

Table 1: Excess over the certified optimum (%) at N = 100,000, mean of three held-out instances, one thread. The last column is the certified solver's time, started from ShuffleSnap; the bottom row is the median time of each method.

distribution	SS fast	BSP-OT 16	SS bal.	BSP-OT 64	SS qual.	RasterFairy	exact (s)
uniform	2.58	44.8	0.84	32.1	0.44	44.8	16.5
Gaussian	<0.001	0.055	<0.001	0.040	<0.001	0.006	55.7
Gaussian mixture	0.013	5.61	0.002	3.74	<0.001	16.7	94.6
90/10 clusters	0.050	0.032	0.012	0.019	<0.001	1.16	214
ring	0.005	1.16	0.001	0.95	<0.001	41.0	114
spiral	0.022	9.96	0.004	8.06	0.001	27.3	120
checkerboard	0.13	49.8	0.057	36.1	0.020	425.2	64.9
heavy tails	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	261
duplicates	0.15	12.3	0.038	5.44	0.013	49.0	1364
near-line	0.010	0.008	0.002	0.002	<0.001	19.4	253
median time (s)	0.77	0.71	2.33	2.85	6.67	3.00	117

5 Quality, time and scale

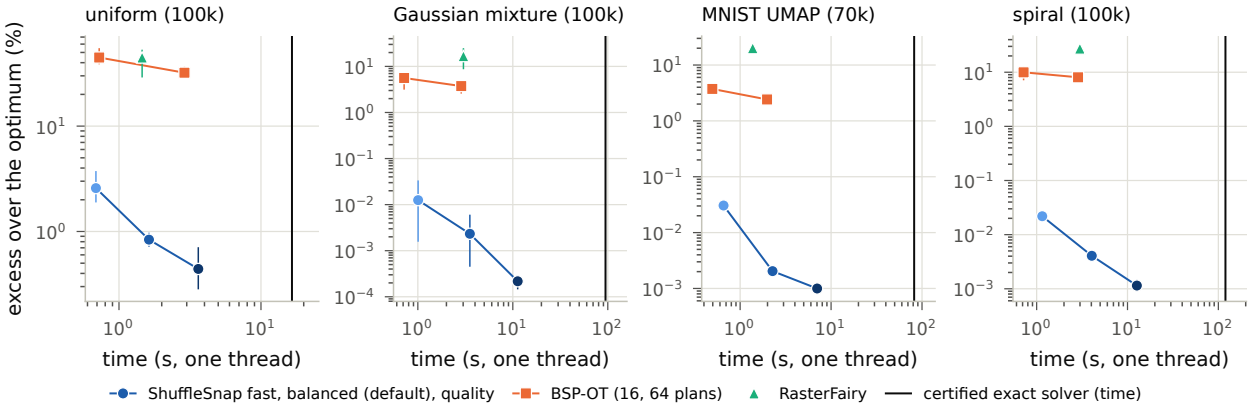


Figure 3: Excess over the certified optimum against time on one core. Points are means over three instances (for MNIST, three runs on the one embedding); bars span their range. The three ShuffleSnap presets use 5×5, 8×8 (default) and 12×12 windows from left to right. The vertical line is the time of the certified exact solver.

On clustered data, the kind of layout that embeddings produce, ShuffleSnap is close to optimal (Figure 3, Table 1). At 10^5 points the default preset is within 0.06% of the optimum on nine of the ten distributions, and within 0.005% on six, in a median 2.3 s on one core. At equal time it beats BSP-OT: the fast preset (0.77 s) is closer to the optimum than BSP-OT with 16 plans (0.71 s) on eight distributions, and the default than 64 plans (2.9 s) on nine; on uniform points they reach 2.6% and 0.84%, against 45% and 32%. BSP-OT is ahead only on the near-line and the 90/10 clusters. RasterFairy is 17–425% above the optimum on seven distributions. Only below 0.12–0.36 s on one core, before ShuffleSnap has finished its coarse levels, is BSP-OT's result the better one.

With 24 threads ShuffleSnap runs 7–14 times faster and returns bit-identical layouts: 10^5 points take 0.09 s with the fast preset and 0.19 s with the default, less than BSP-OT's 0.13 s and 0.41 s. Time grows nearly linearly with N; 10^6 uniform points take 21 s on one core and 1.5 s on 24 threads with the default, using about 55 bytes per point, against about 250 and 670 for BSP-OT and RasterFairy. A billion points converged in 1 h 43 min using 53 GiB, passing a six-hour run of an earlier implementation after 44 minutes; a single-plan BSP-OT ended 8.2% above. The exact solver scales worse but further than we expected. Seeded with ShuffleSnap's answer, it proved optimality at 10^5 points in 17 s to 23 min on one core, with a median near two minutes, and at 10^6 uniform points in 11 min.

What mattered in the design: larger windows are the main quality knob, the coarse levels are essential, without shifted tilings the method fails outright, and the $\sqrt{2}$ schedule is best or close to best among the stride ratios we tried. Structured starts, extra polishing at stride 2, merging independent runs and a Jonker–Volgenant window solver did not pay off. Starting each window solve from closed-form transport duals made it 2–5 times faster, which buys larger windows at equal time.

Masks and surplus cells. With 20,000 points on disks and rings, the default preset stays within 0.07% of the optimum (fast: 0.14%), where BSP-OT is 2.5–9.3% and RasterFairy 11–54% above it. Surplus cells, where a method must also choose which cells stay empty, hurt the comparators most: with 10% surplus, BSP-OT's partial matching is 18–109% above the optimum and RasterFairy, which removes surplus cells at random, 72–350%, while the default stays within 0.91%. Concave masks and fewer surplus cells are harder (Section 7).

6 Real embeddings: displacement is not neighbourhood

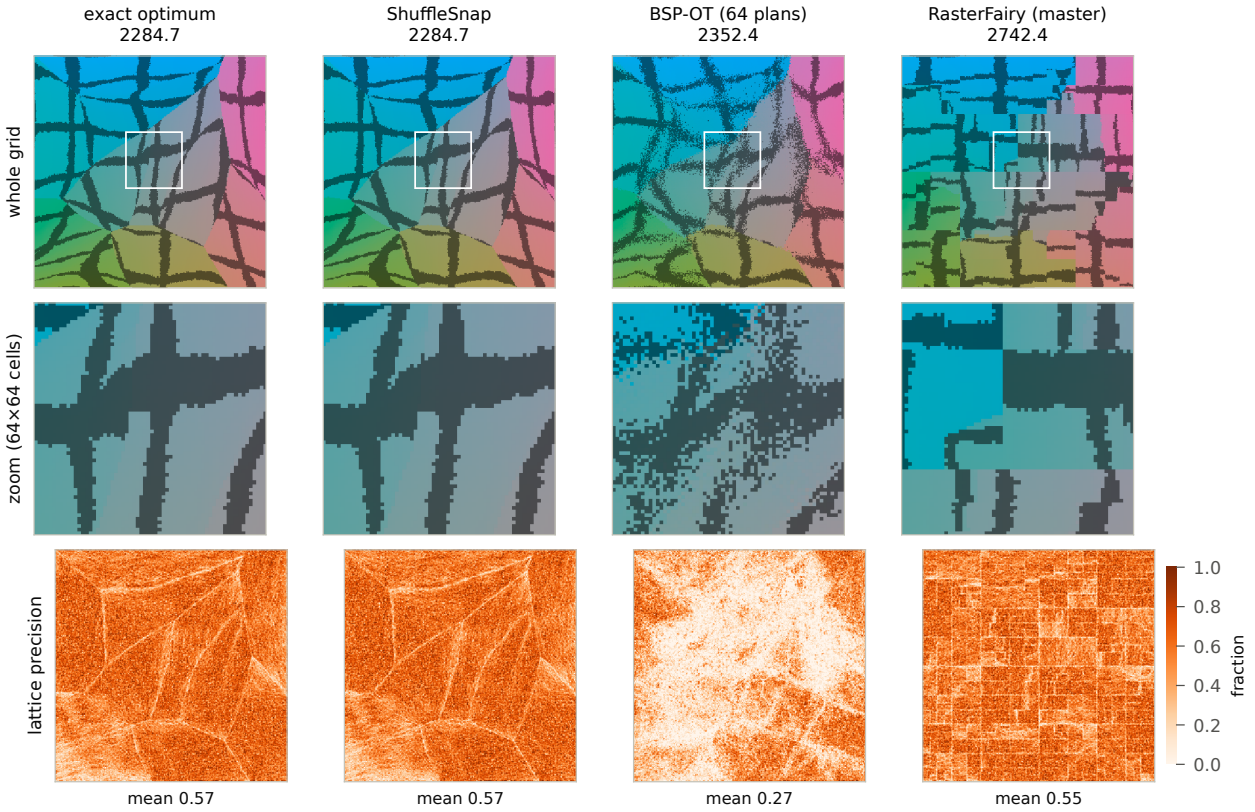


Figure 4: All 70,000 MNIST digits on a 265×265 grid. Colors and graph-paper lines follow each point's input position; numbers are the mean squared displacement in cells^2. The bottom row shows lattice precision, the fraction of a point's occupied neighbour cells that hold one of its 8 nearest input neighbours.

On both embeddings the default preset is within 0.002% of the certified optimum, and its layout is visually indistinguishable from the optimal one: the graph-paper lines bend in the same places (Figure 4). The two comparators fail in opposite ways. BSP-OT is within 2.4% (MNIST) and 6.7% (Fashion-MNIST) of the optimal displacement, yet it halves both neighbourhood measures (the fraction of each point's 10 nearest input neighbours among its 10 nearest grid neighbours is 0.31–0.33, against 0.61–0.65 for the optimum): its merged partitions follow the optimal map on average but scramble points locally, which frays the lines. RasterFairy moves points 20–25% more than necessary, but keeps local order inside its slices and breaks it only along their seams, so its neighbourhood measures stay close to the optimum's. Even the optimum tears the layout along cluster boundaries, visible as pale lines in the bottom row. Minimizing displacement and preserving neighbourhoods are different goals; neither implies the other.

7 Where it fails

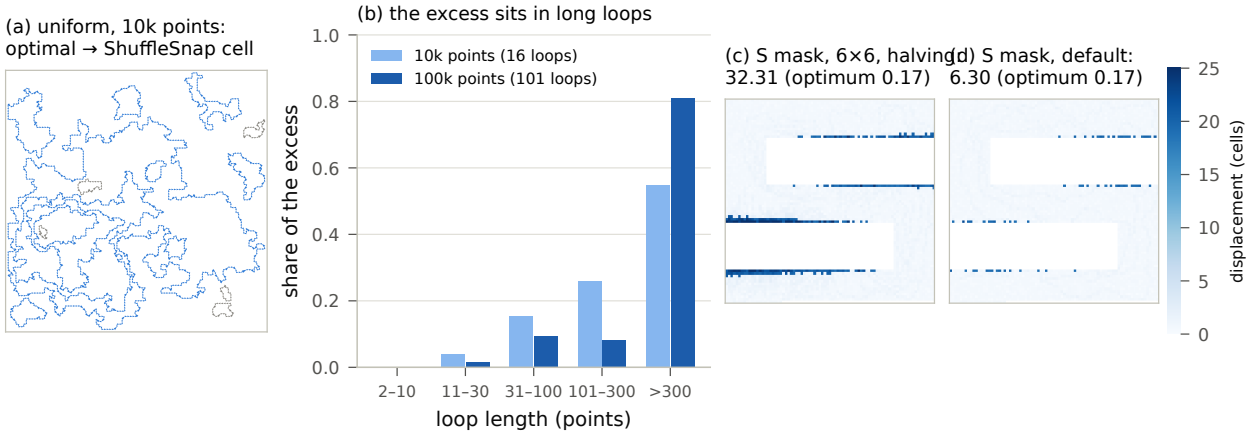


Figure 5: (a, b) Where ShuffleSnap and the optimum disagree on uniform points, the difference is a few long closed loops of one-cell moves (loops longer than 30 points in blue), and at 10^5 points loops of more than 300 points carry most of the excess. (c, d) Points drawn uniformly inside a blocky S-shaped mask, whose optimum moves every point by less than a cell: 6×6 windows with halving strides route points across the concave bays, where they line up along the edges; the default preset leaves fewer such lines. Numbers are mean squared displacements.

ShuffleSnap is a local search, and its failures follow from that. The excess sits in long loops in which every point is shifted by about one cell (Figure 5a,b). Such a loop is optimal inside every window that sees only part of it, and a strided window cannot shift a point by one cell. We built inputs on which the method never moves while the optimum costs zero, so its relative excess is unbounded. Uniform points are the hardest common input: the default's excess grows with N, from 0.1% at 10^3 points to 0.84% at 10^5 and 1.3% at 10^6. On masks with concave bays, coarse schedules route points across the bays (Figure 5c,d). The default reduces this, and with uniform points on the S mask it is 1.4% above the optimum (BSP-OT: 9.3%). A mask can even cut two usable cells off from each other, when no scheduled window contains both. Grids with a few surplus cells are hard too, because the solver must also choose where the holes go (3.1% with less than a row of them).

When to use it. For interactive layouts of 10^5 points, the default preset gives near-optimal results in about 0.2 s on a multicore machine (fast: 0.09 s). When a proven optimum is worth minutes, the certified solver started from ShuffleSnap is the better choice. Below 0.12–0.36 s on one core, BSP-OT gives the best available result. If the position of empty cells does not matter, an exact-fit mask is easier than surplus cells for every method. The package is on PyPI as `shufflesnap`; raw results and figure scripts are in a separate repository.

Limits of this study. All measurements come from one workstation, CPU only; BSP-OT also has a GPU implementation that we did not test. The objective is squared Euclidean displacement in 2D, the real data are two UMAP embeddings with one parameter setting, and each comparator ran at representative settings, not at every setting.

References

[1] Baptiste Genest, Nicolas Bonneel, Vincent Nivoliers, and David Coeurjolly. BSP-OT: Sparse transport plans between discrete measures in loglinear time. *ACM Transactions on Graphics*, 44(6):1–15, December 2025. doi: 10.1145/3763281. URL https://doi.org/10.1145/3763281. Proc. SIGGRAPH Asia 2025.

[2] R. Jonker and A. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987. doi: 10.1007/BF02278710. URL https://doi.org/10.1007/BF02278710.

[3] Mario Klingemann. RasterFairy. https://github.com/Quasimondo/RasterFairy, 2015. URL https://github.com/Quasimondo/RasterFairy. GitHub repository created 2016-01-22; latest PyPI release of `rasterfairy` is 1.0.6 (2019-04-29). Accessed 2026-09-23.

[4] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109. URL https://doi.org/10.1002/nav.3800020109.

[5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. URL https://doi.org/10.1109/5.726791.

[6] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction, 2018. URL https://arxiv.org/abs/1802.03426.

[7] Laurent Perron and Vincent Furnon. OR-Tools. https://developers.google.com/optimization/, 2026. URL https://developers.google.com/optimization/. Version v9.15, released 2026-01-12. Accessed 2026-09-23.

[8] Gabriel Peyré and Marco Cuturi. Computational optimal transport with applications to data sciences. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019. doi: 10.1561/2200000073. URL https://doi.org/10.1561/2200000073.

[9] Bernhard Schmitzer. A sparse multiscale algorithm for dense optimal transport. *Journal of Mathematical Imaging and Vision*, 56(2):238–259, 2016. doi: 10.1007/s10851-016-0653-9. URL https://doi.org/10.1007/s10851-016-0653-9.

[10] Donald L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32, 1959. doi: 10.1145/368370.368387. URL https://doi.org/10.1145/368370.368387.

[11] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008. URL https://jmlr.org/papers/v9/vandermaaten08a.html.

[12] A. Volgenant. Linear and semi-assignment problems: A core oriented approach. *Computers & Operations Research*, 23(10):917–932, 1996. doi: 10.1016/0305-0548(96)00010-X. URL https://doi.org/10.1016/0305-0548(96)00010-X.

[13] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. URL https://arxiv.org/abs/1708.07747.